

Project Title: Linux Hardening Audit Tool

Name: **Ritika Karak**

Domain: **Cyber Security**

1. Introduction

In standard enterprise architectures, Linux servers handle core infrastructural data, running applications, databases, and network-exposed web services. Ensuring that these hosts are structurally insulated from cyber attacks is a foundational security mandate. Linux Hardening refers to the targeted minimization of a system's attack surface through specific structural adjustments, disabling unnecessary network packages, and maintaining rigid configuration control. Since scanning hundreds of parameters manually introduces critical delays and human oversights, this project establishes a dedicated automation tool designed to independently check, evaluate, and rate system configurations against vital security configurations.

2. Abstract

This project introduces a modular, lightweight Python-based compliance tool built to evaluate essential system integrity parameters on a Linux endpoint. Operating directly on the host machine, the script focuses on auditing high-value security markers: specific discretionary access controls on central credential files (`/etc/passwd` and `/etc/shadow`) alongside the operative state of the local network filtering system (Uncomplicated Firewall or UFW). By parsing OS components, checking file states, and capturing command outputs, the script scores each parameter as a PASS or FAIL. The tool generates a real-time compliance dashboard that provides a clear metric of host security health, helping administrators identify and fix vulnerabilities quickly.

3. Tools Used

To preserve full operational autonomy across minimal Linux deployments, the tool relies exclusively on native system facilities without external Python package dependencies:

- **Core Language:** Python 3 (For parsing structures and formatting output dashboards).

- **System Modules:** * `os`: Leveraged to check path configurations, extract file metadata, and parse octal file permissions.
 - `subprocess`: Employed to run low-level terminal binaries securely and redirect standard error output streams.
- **Testing Platform:** Kali Linux Environment configured on an Oracle VirtualBox Hypervisor.

4. Steps Involved in Building the Project

The development cycle was conducted linearly through several practical milestones:

1. **Defining the Baselines:** Conducted initial structural research to identify key configuration parameters. Focused on checking the isolation of access credentials (`/etc/shadow`) and validating that network edge traffic filtering is functional.
2. **Environment Integration:** Set up a clean directory structure within Kali Linux. Since checking kernel-level configurations requires elevated execution privileges, administrative boundaries (`sudo`) were mapped out for testing.
3. **Module 1 - Permission Auditing:** Programmed individual check mechanisms utilizing `os.stat()`. The function pulls the file mode bits, isolates the permission triplet, converts it to an octal string, and evaluates it against safe defaults (e.g., matching `644` for globally readable text structures and verifying locked permission states for sensitive files).
4. **Module 2 - Network State Inspection:** Structured execution pipelines via `subprocess.check_output()`. The script issues low-level system calls to inspect `ufw status`, actively capturing string outputs to find the explicit sub-string declaration "Status: active".
5. **Module 3 - Grading Framework & Testing:** Implemented a points-allocation mechanism where each passing metric updates a global counter. Exception blocks (`try-except`) were mapped out around network functions to handle execution anomalies gracefully without stopping the script entirely.

5. Conclusion

The implementation of the automated Linux Hardening Audit Tool proves that foundational host defense assessments can be simplified into rapid, single-click operations. By shifting manual system discovery into automated code loops, the room for human oversight is eliminated. The execution runtime successfully exposes security variances—such as identifying a relaxed `/etc/shadow` permission state (`640`) on the target machine and lowering the total compliance rating to `66.67%` as a clear security warning. Future improvements can scale this framework by adding automated mitigation modules to fix discovered vulnerabilities instantly, as well as integrating simple remote syslog notifications.

Python code:

```
ritika@kali: ~/internship_elavorate
Session Actions Edit View Help
import os
import subprocess

def check_file_permissions(filepath, expected_perm):
    if os.path.exists(filepath):
        stat = os.stat(filepath)
        current_perm = oct(stat.st_mode)[-3:]
        if current_perm == expected_perm:
            return f"[PASS] {filepath} has secure permissions ({current_perm}).", 20
        else:
            return f"[FAIL] {filepath} permissions are insecure ({current_perm}). Expected: {expected_perm}", 0
    return f"[WARN] {filepath} not found.", 0

def check_ufw_status():
    try:
        output = subprocess.check_output(["sudo", "ufw", "status"], stderr=subprocess.STDOUT).decode()
        if "Status: active" in output:
            return "[PASS] Firewall (UFW) is ACTIVE.", 20
        else:
            return "[FAIL] Firewall (UFW) is INACTIVE.", 0
    except Exception:
        return "[WARN] Could not determine UFW status.", 0

def main():
    print("="*50)
    print("      LINUX HARDENING AUDIT TOOL REPORT")
    print("="*50)

    total_score = 0
    max_score = 60

25,1 Top
```

```
ritika@kali: ~/internship_elavorate
Session Actions Edit View Help
print("="*50)
print("      LINUX HARDENING AUDIT TOOL REPORT")
print("="*50)

total_score = 0
max_score = 60

res1, score1 = check_file_permissions("/etc/shadow", "000")
print(res1)
total_score += score1

res2, score2 = check_file_permissions("/etc/passwd", "644")
print(res2)
total_score += score2

res3, score3 = check_ufw_status()
print(res3)
total_score += score3

print("-" * 50)
compliance_percentage = (total_score / max_score) * 100
print(f"TOTAL COMPLIANCE SCORE: {compliance_percentage:.2f}%")
print("="*50)

if __name__ == "__main__":
    main()

~
~
30,1 Bot
```

Output:

```
ritika@kali: ~/internship_elaborate
Session Actions Edit View Help
(ritika@kali)-[~]
$ ls
arpan Desktop email internship_elaborate Pictures ritika.sh test
Augnik-Guha.txt Documents IMEI-TRACKER k Public shell_practice Videos
CamPhish Downloads intern_scan.txt Music ritika Templates zphisher

(ritika@kali)-[~]
$ cd interb
cd: no such file or directory: interb

(ritika@kali)-[~]
$ cd internship_elaborate

(ritika@kali)-[~/internship_elaborate]
$ vi audit.tool.py

(ritika@kali)-[~/internship_elaborate]
$ sudo python3 audit.tool.py
[sudo] password for ritika:

=====
LINUX HARDENING AUDIT TOOL REPORT
=====
[FAIL] /etc/shadow permissions are insecure (640). Expected: 000
[PASS] /etc/passwd has secure permissions (644).
[PASS] Firewall (UFW) is ACTIVE.
=====
TOTAL COMPLIANCE SCORE: 66.67%
=====

(ritika@kali)-[~/internship_elaborate]
$
```